Проектирование ИС: сервис студенческих знакомств CAMPUS (IDEF0, IDEF1X, DFD, IDEF3, FURPS+, UML)

9 ноября 2026 · Команда CAMPUS · УрФУ · расширенный уровень

1. Описание предметной области

Предметная область — сервис знакомств и совместных активностей для студентов вузов Екатеринбурга. Сервис работает как Telegram Mini App: студент входит через Telegram, подтверждает статус студента кодом на корпоративную почту вуза, заполняет анкету и получает ленту анкет. После взаимного лайка открывается чат, где можно договориться о встрече. Команда сервиса вместе с партнёрами-площадками (антикафе, кофейни) проводит офлайн-мероприятия; билеты и подписка Premium оплачиваются через платёжный сервис. Жалобы рассматривают модераторы. Учебная версия рассчитана на пилот в одном вузе (УрФУ) и один сервер; оплата работает в тестовом режиме. Цифры и детали — допущения.

Участники (роли):

Роль	Что делает в системе
Студент	Входит, подтверждает статус, заполняет анкету, ставит лайки, переписывается, предлагает встречи, жалуется, записывается на мероприятия, оплачивает билеты и Premium
Модератор	Рассматривает жалобы, принимает решение: отклонить, предупредить или заблокировать
Организатор (команда CAMPUS)	Согласует площадку с партнёром, публикует мероприятие, ведёт список участников
Партнёр (площадка)	Предоставляет место и условия, получает список участников
Администратор (команда CAMPUS)	Настраивает тарифы и правила, смотрит отчёты
Платёжный сервис (внешняя система)	Принимает оплату и присылает уведомление о результате
Система (по расписанию)	Снимает неоплаченные брони, завершает истёкший Premium, считает отчёты

Основные объекты (сущности): пользователь, анкета, фото, интерес, лайк, совпадение, сообщение, встреча, жалоба, партнёр, мероприятие, билет, платёж, подписка Premium.

Бизнес-правила:

1. Пользоваться сервисом можно только с 18 лет; возраст считается по дате рождения при регистрации.
2. Отметка «студент подтверждён» ставится только после ввода кода, отправленного на почту в домене вуза. Без отметки анкета в ленту не попадает.
3. В ленту попадает только заполненная анкета: не меньше 1 фото и 3 интересов.
4. Чат открывается только после взаимного лайка. Бесплатно — до 30 лайков в сутки, с Premium — без лимита.
5. Студент может скрыть свою анкету от студентов своего института.
6. Анкета, на которую пожаловались три разных пользователя, скрывается из ленты до решения модератора; решение принимается в течение 24 часов.
7. Билет и Premium действуют только после подтверждения оплаты платёжным сервисом; неоплаченная бронь снимается через 30 минут. Premium действует 30 дней и стоит 349 ₽.
8. Точная геолокация не собирается и не хранится.

1.1. Дерево задач



Дерево задач сервиса CAMPUS

Логическое описание. Корень дерева (уровень 0) — деятельность сервиса в целом: помочь студентам знакомиться и встречаться. Она распадается на пять ветвей (уровень 1), и каждая ветвь становится блоком функциональной модели IDEF0 с тем же номером:

- **Работа с пользователями (1)** — без учётной записи, подтверждённого статуса и анкеты остальные ветви работать не могут, поэтому она первая.
- **Подбор и знакомства (2)** — центральная ветвь: использует анкеты из ветви 1 и даёт совпадения, переписку и встречи.
- **Безопасность и модерация (3)** — получает жалобы из ветви 2 и возвращает решение обратно: статус пользователя в ветвь 1 и исключение анкеты из ленты в ветвь 2.

- **Мероприятия (4)** — вторая форма знакомства: события с партнёрами, на которые записываются студенты.
- **Оплата и отчётность (5)** — последний блок верхнего уровня: проверка оплаты. Его результат (статус оплаты) возвращается в ветвь 4 (билет подтверждён) и в ветвь 2 (Premium снимает лимит лайков). Лист 53 «Отчёты» стоит в этой ветви последним, но в обработку платежа не входит: он отдельно использует уже накопленные данные за период.

Нумерация. Используется многозначная (десятичная) нумерация: уровень 1 — 1…5, уровень 2 — 11, 12, …, 53 (первая цифра — номер ветви, вторая — номер задачи в ветви). Номер узла дерева совпадает с номером блока IDEF0 без буквы A: задача 52 — это блок A52. Такой порядок нумерации принят в стандарте IDEF0 (Р 50.1.028-2001, FIPS 183) и позволяет пройти от листа дерева к диаграмме и обратно.

Правило «один лист — одна ветвь». Каждая задача встречается в дереве один раз. Вход в систему есть только в листе 11, проверка оплаты — только в листе 52; ветвь 4 лишь формирует заказ билета и читает статус оплаты, но сама оплату не проверяет. Листья — это места, где появляется хранилище данных (стрелка «→ D…» под названием): их таблицы и есть база данных (раздел 7) и хранилища DFD (раздел 8).

1.2. Результаты задач и движение информации

Для каждого листа указано, какая информация в нём формируется (не документ, а конкретные данные), кому она передаётся дальше и где хранится. Эти результаты становятся стрелками IDEF0 и потоками DFD.

Задача	Результат (информация)	Куда передаётся	Хранилище
11 Регистрация и вход	Учётная запись, сессия, статус пользователя	12, 21	D1.1 Пользователи
12 Подтверждение статуса студента	Отметка «студент подтверждён»	13, 21	D1.1 Пользователи
13 Заполнение анкеты	Анкета: фото, интересы, цель знакомства	21	D1.2 Анкеты, D1.3 Фото
21 Формирование ленты	Лента анкет для студента	22, студенту	— (читает D1.2, D2.1)
22 Лайки и совпадения	Лайк, совпадение; оценённые анкеты	23, 21, 53	D2.1 Лайки и совпадения
23 Переписка	Сообщения; жалоба из чата	24, 31	D2.2 Сообщения
24 Согласование встречи	Предложение и ответ: место, время, статус	студентам, 53	D2.3 Встречи
31 Приём жалоб	Жалоба в очереди; скрытие анкеты при 3 жалобах	32, 21	D3 Жалобы
32 Рассмотрение жалобы	Решение модератора	33	D3 Жалобы
33 Применение решения	Статус пользователя, уведомления	11, 21, студентам	D1.1 Пользователи

Задача	Результат (информация)	Куда передаётся	Хранилище
41 Согласование с партнёром	Условия площадки: дата, вместимость, цена	42	D4.1 Партнёры
42 Публикация мероприятия	Карточка мероприятия, афиша	43, студентам	D4.2 Мероприятия
43 Запись участников	Бронь (заказ билета); билет; список участников	51, партнёру	D4.3 Билеты
51 Создание платежа	Платёж «ожидает», ссылка на оплату	52, студенту	D5.1 Платежи
52 Проверка оплаты	Статус оплаты; подписка Premium	43, 22, 53	D5.1 Платежи, D5.2 Подписки
53 Формирование отчётов	Отчёт за период	команде, партнёрам	— (читает D1–D5)

Видно, как информация возвращается между уровнями: со второго уровня ветви 4 (лист 43) заказ уходит в ветвь 5, проверка оплаты (52) возвращает статус обратно в лист 43, и только тогда студент получает билет. Так же решение модератора (32 → 33) возвращается в ветвь 1 (статус пользователя) и в ветвь 2 (лента).

1.3. Матрица ответственности

Матрица построена по листьям дерева и показывает, кто выполняет каждую задачу. Обозначения: **И** — исполняет, **У** — участвует (предоставляет данные), **К** — контролирует или утверждает, **Н** — получает результат. Столбцы с буквой И становятся актёрами диаграммы вариантов использования (раздел 11.1) по таким правилам: «Студент», «Модератор» и «Платёжный сервис» переходят как есть; «Организатор» и «Админ» — это сотрудники команды сервиса, поэтому на UML они объединены в одного актёра «Команда CAMPUS»; «Система» — это сам сервис (граница системы на UML), а её задачи по расписанию (снятие неоплаченных броней в задачах 43/52 и завершение Premium в задаче 52) запускает системный актёр «Таймер» — планировщик на сервере. «Партнёр» в матрице только участвует (У) и получает результат (Н), но сам в системе ничего не выполняет, поэтому актёром UML не стал.

Задача	Студент	Модератор	Организатор	Партнёр	Админ	Платёжный сервис	Система
11 Регистрация и вход	И						У
12 Подтверждение статуса	И						У
13 Заполнение анкеты	И						У
21 Формирование ленты	Н						И
22 Лайки и совпадения	И						У
23 Переписка	И						У
24 Согласование встречи	И						У
31 Приём жалоб	И	Н					У

Задача	Студент	Модератор	Организатор	Партнёр	Админ	Платёжный сервис	Система
32 Рассмотрение жалобы		И			К		
33 Применение решения	Н	И					У
41 Согласование с партнёром			И	У	К		
42 Публикация мероприятия	Н		И				У
43 Запись участников	И		Н	Н			У
51 Создание платежа	И					У	У
52 Проверка оплаты	Н		Н			У	И
53 Формирование отчётов			Н	Н	Н		И

Из матрицы следует: студент исполняет 9 задач из 16, поэтому интерфейс строится вокруг него; модератор нужен только в ветви 3; платёжный сервис участвует в двух задачах ветви 5 и в программе становится внешней системой с API; задачи 21, 52 и 53 выполняются автоматически, без участия людей.

2. Цель, точка зрения и границы модели

Параметр	Значение
Название модели	«Деятельность сервиса знакомств CAMPUS»
Нотации	IDEF0 (функциональная модель), IDEF1X (модель данных), DFD (потоки данных, Гейн–Сарсон), IDEF3 (сценарии процессов), UML 2.5
Цель моделирования	Описать функции сервиса и потоки информации между ними, чтобы определить требования к ИС и сформировать техническое задание на MVP
Точка зрения	Команда (администратор) сервиса
В границах модели	Регистрация и подтверждение студента, анкеты и лента, лайки, совпадения, чат и встречи, жалобы и модерация, мероприятия с партнёрами, оплата билетов и Premium, отчёты
За границами модели	Реальный эквайринг (только тестовый режим), мобильные приложения в магазинах, рекомендации на машинном обучении, геолокация, реклама, бухгалтерия партнёров

Нумерация блоков — по стандарту IDEF0: A-0 (контекст), A1…A5 (первый уровень), A11…A53 (второй уровень — декомпозиция каждого блока). Стрелки ICOM: **I** — вход (слева), **C** — управление (сверху), **O** — выход (справа), **M** — механизм (снизу).

3. Предположения по сценариям и нагрузке

Семь сценариев покрывают все блоки модели; основная нагрузка — вечерний просмотр ленты (блок A2).

3.1. Сценарии

№	Сценарий	Участник	Ход	Результат	Блоки
С1	Регистрация и подтверждение	Новый студент	Вход через Telegram → дата рождения (18+) → почта вуза → код → анкета	Анкета в ленте	А1
С2	Знакомство (основной)	Студент	Лента → лайк → взаимный лайк → совпадение → чат → предложение встречи	Согласованная встреча	А2
С3	Жалоба	Студент, модератор	Жалоба → при 3 жалобах анкета скрыта → рассмотрение → решение	Статус пользователя	А3
С4	Мероприятие	Организатор, партнёр	Согласование площадки → публикация → запись студентов	Мероприятие с участниками	А4
С5	Покупка билета	Студент, платёжный сервис	Бронь на 30 мин → платёж → уведомление → проверка → билет или снятие брони	Оплаченный билет	А4, А5
С6	Покупка Premium	Студент	Оплата 349 ₽ → проверка → Premium на 30 дней → лимит лайков снят	Активный Premium	А5, А2
С7	Отчёт	Администратор	Выбор периода → регистрации, совпадения, удержание D7, оплаты	Отчёт за период	А5

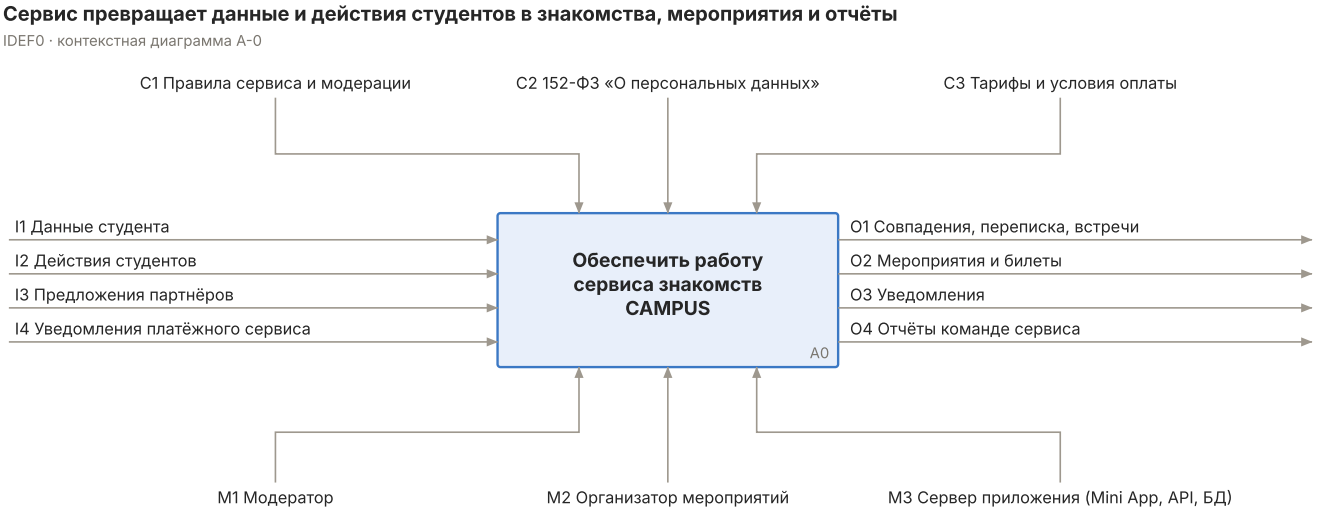
3.2. Предположения о нагрузке

Учебный масштаб: пилот в одном вузе, один сервер; все числа — допущения.

Показатель	Значение	Как получено
Зарегистрированные студенты	1 000	допущение для пилота в УрФУ
Активные за день (DAU)	300	30% зарегистрированных
Просмотры анкет в сутки	≈ 12 000	300 × 40 анкет
Лайки в сутки	≈ 3 000	25% просмотров
Совпадения в сутки	≈ 150	5% лайков взаимны
Сообщения в сутки	≈ 3 000	300 активных пар × 10 сообщений
Одновременно онлайн в пик (21:00–23:00)	≈ 60	20% DAU
Запросы к API в пик	≈ 5 в секунду	60 пользователей × 1 запрос в 12 с
Объём фото	≈ 1 ГБ	1 000 × 3 фото × 300 КБ
Оплаты	≈ 5 в сутки	Premium 3% (30 в месяц) + 4 мероприятия × 30 билетов

Выводы для проектирования: нагрузка на один сервер небольшая; узкое место не производительность, а плотность аудитории (нужны анкеты в ленте). Самый частый запрос — чтение ленты, поэтому на таблицах анкет и лайков нужны индексы, а лента отдаётся страницами. Фото лежат в объектном хранилище, в базе — только ключи к ним.

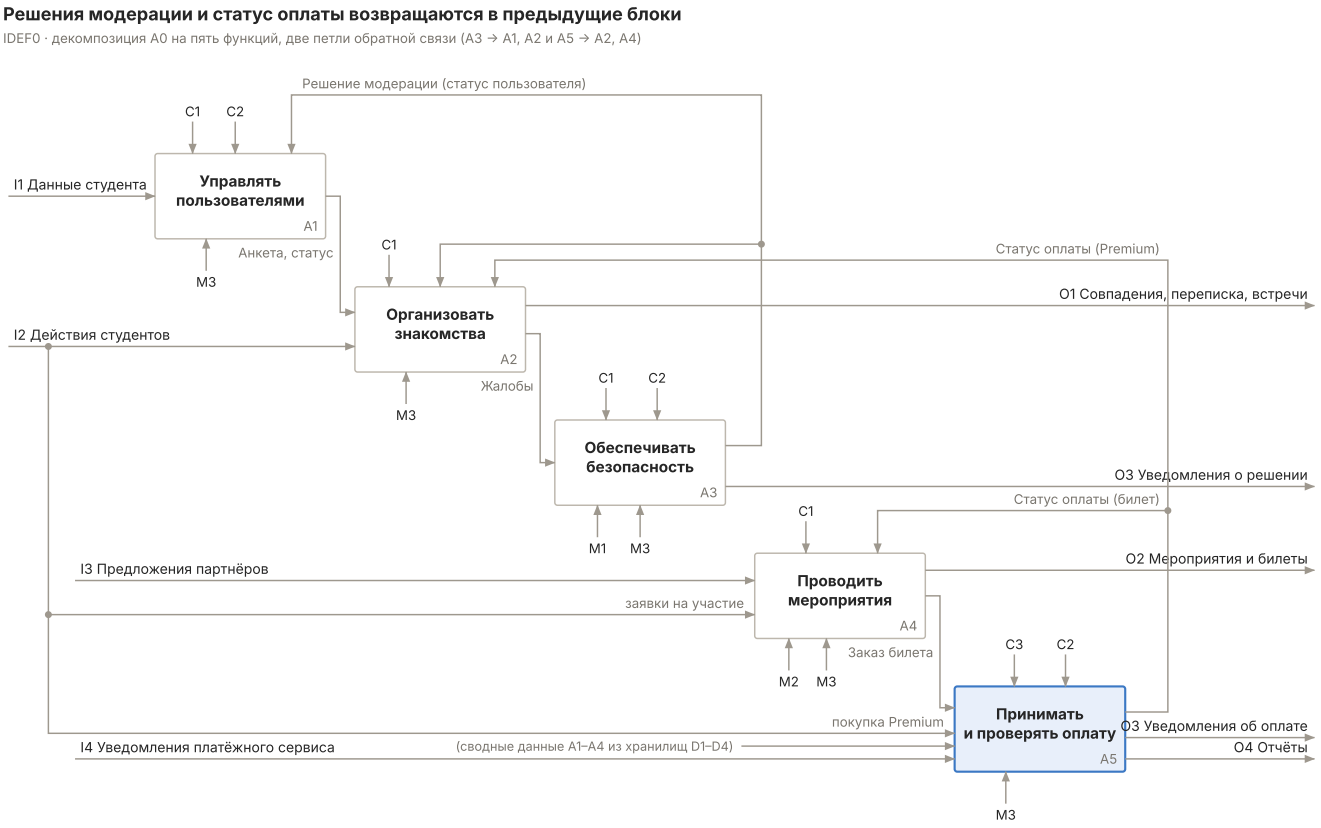
4. Контекстная диаграмма A-0



IDEF0 A-0 · контекстная диаграмма · точка зрения: команда сервиса

Логическое описание. Контекстная диаграмма показывает весь сервис одним блоком A0 и его связи с внешним миром. Слева входит то, что сервис перерабатывает: данные студента для регистрации и анкеты (I1), действия студентов — лайки, сообщения, жалобы, заявки и покупки (I2), предложения партнёров-площадок (I3) и уведомления платёжного сервиса (I4). Сверху деятельность ограничивают правила сервиса и модерации (C1), закон о персональных данных (C2) и тарифы (C3). Снизу — кто выполняет работу: модератор (M1), организатор мероприятий (M2) и сервер приложения с базой данных (M3). Справа — результат: совпадения, переписка и встречи (O1), мероприятия и билеты (O2), уведомления (O3) и отчёты команде (O4).

5. Декомпозиция A0



IDEF0 A0 · декомпозиция на пять функций

Логическое описание. Пять блоков расположены лесенкой по доминированию. A1 готовит анкету и статус пользователя, без которых A2 не может построить ленту. A2 выдаёт совпадения и переписку (O1) и передаёт жалобы в A3. A3 возвращает решение модерации вверх как управление: в A1 (статус: активен или заблокирован) и в A2 (анкета исключается из ленты) — первая петля обратной связи. A4 получает предложения партнёров и заявки студентов и передаёт заказ билета в A5. A5 — последний блок, проверка оплаты: статус оплаты возвращается управлением в A4 (бронь становится билетом) и в A2 (Premium снимает лимит лайков) — вторая петля. Для отчётов A5 получает ещё и сводные данные о пользователях, знакомствах и мероприятиях, накопленные блоками A1–A4 в хранилищах D1–D4. Они показаны туннельной стрелкой (в скобках): на IDEF0 нет хранилищ, а сам путь данных виден на DFD (раздел 8.6). Модератор (M1) работает только в A3, организатор (M2) — только в A4, остальное выполняет сервер приложения (M3).

Блок	Функция	Вход (I)	Управление (C)	Выход (O)	Механизм (M)
A1	Управлять пользователями	I1 Данные студента	C1, C2, решение модерации (из A3)	Анкета и статус пользователя (в A2)	M3
A2	Организовать знакомства	Анкета (из A1), I2 Действия студентов	C1, решение модерации (A3), статус Premium (A5)	O1 Совпадения, переписка, встречи; жалобы (в A3)	M3

Блок	Функция	Вход (I)	Управление (C)	Выход (O)	Механизм (M)
A3	Обеспечивать безопасность	Жалобы (из A2)	C1, C2	Решение модерации (в A1, A2), O3	M1, M3
A4	Проводить мероприятия	I3 Предложения партнёров, заявки (I2)	C1, статус оплаты (A5)	Заказ билета (в A5), O2 Мероприятия и билеты	M2, M3
A5	Принимать и проверять оплату	Заказ билета (A4), покупка Premium (I2), I4, (сводные данные D1–D4)	C3 Тарифы, C2	Статус оплаты (в A2, A4), O3, O4 Отчёты	M3

6. Декомпозиция второго уровня (A1–A5)

Каждый блок A0 развёрнут на отдельной диаграмме (расширенный уровень). Стрелки на границе повторяют стрелки родительского блока на A0, но коды ICOM у них локальные — расшифровка в подписи под каждой диаграммой. Управление и механизмы показаны короткими отрезками у блоков, чтобы линии не пересекались.

6.1. Декомпозиция A1 «Управлять пользователями»



IDEF0 A1 · декомпозиция «Управлять пользователями»

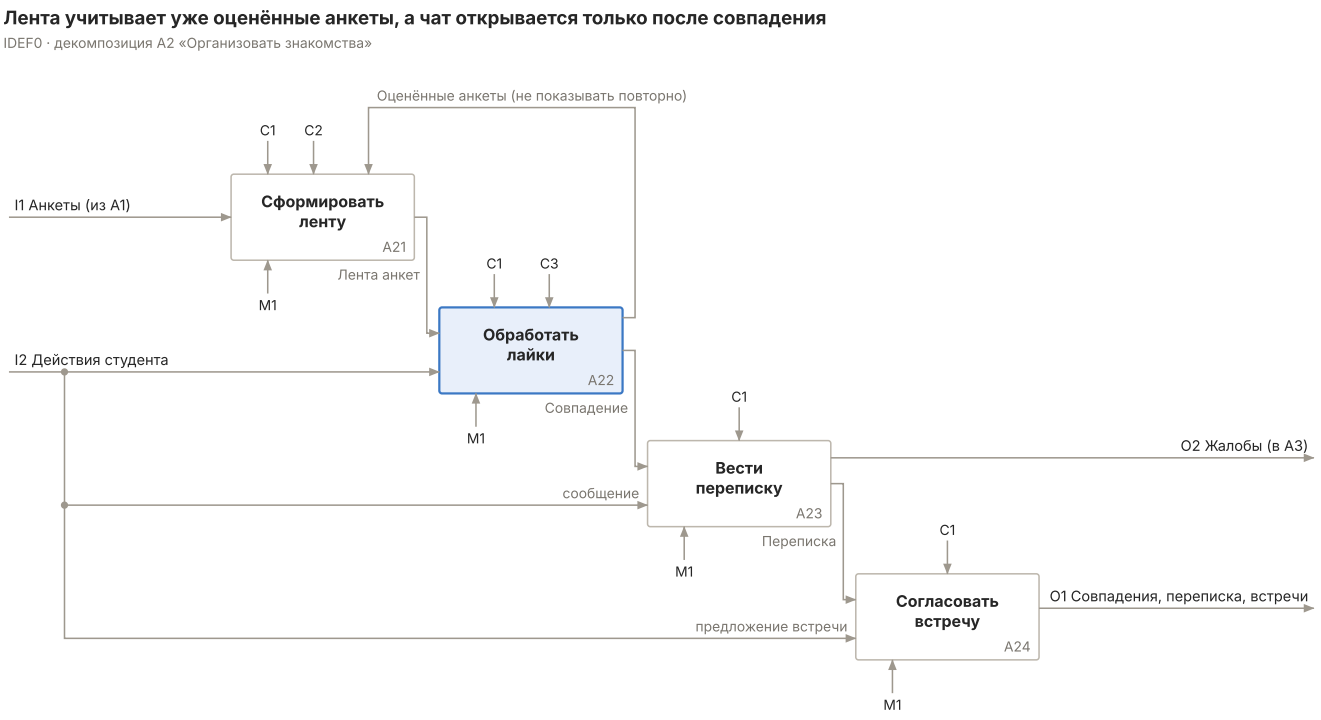
Узел A1 · I1 — данные студента (Telegram-аккаунт, дата рождения, почта вуза, фото, интересы) · C1 — правила сервиса · C2 — 152-ФЗ · C3 — решение модерации (из A3) · O1 — анкета (в A2) · O2 — статус пользователя (в A2) · M1 — сервер приложения

Логическое описание. Блок покрывает сценарий C1. Данные Telegram и дата рождения превращаются в учётную запись (A11) — с согласием на обработку персональных данных (C2) и только для 18+ (C1). Если модератор заблокировал пользователя (C3), вход запрещается, а статус пользователя выходит наружу (O2). Учётная запись и почта вуза попадают в A12:

студент получает одноразовый код и вводит его, после чего ставится отметка «студент». Только с этой отметкой A13 принимает фото и интересы и выпускает анкету (O1), которая уходит в ленту (A2).

Блок	Вход	Управление	Выход	Механизм
A11 Зарегистрировать и авторизовать	I1 Telegram-аккаунт, дата рождения	C1 (18+), C2 (согласие), C3	Учётная запись, O2 Статус	M1
A12 Подтвердить статус студента	Учётная запись, I1 почта вуза и код	C2	Отметка «студент»	M1
A13 Заполнить анкету	Отметка «студент», I1 фото, интересы, цель	C1 (≥ 1 фото, ≥ 3 интересов)	O1 Анкета	M1

6.2. Декомпозиция A2 «Организовать знакомства»



IDEF0 A2 · декомпозиция «Организовать знакомства»

Узел A2 · I1 — анкеты (из A1) · I2 — действия студента · C1 — правила (лимит 30 лайков, публичное место встречи) · C2 — решение модерации (из A3) · C3 — статус Premium (из A5) · O1 — совпадения, переписка, встречи · O2 — жалобы (в A3) · M1 — сервер приложения

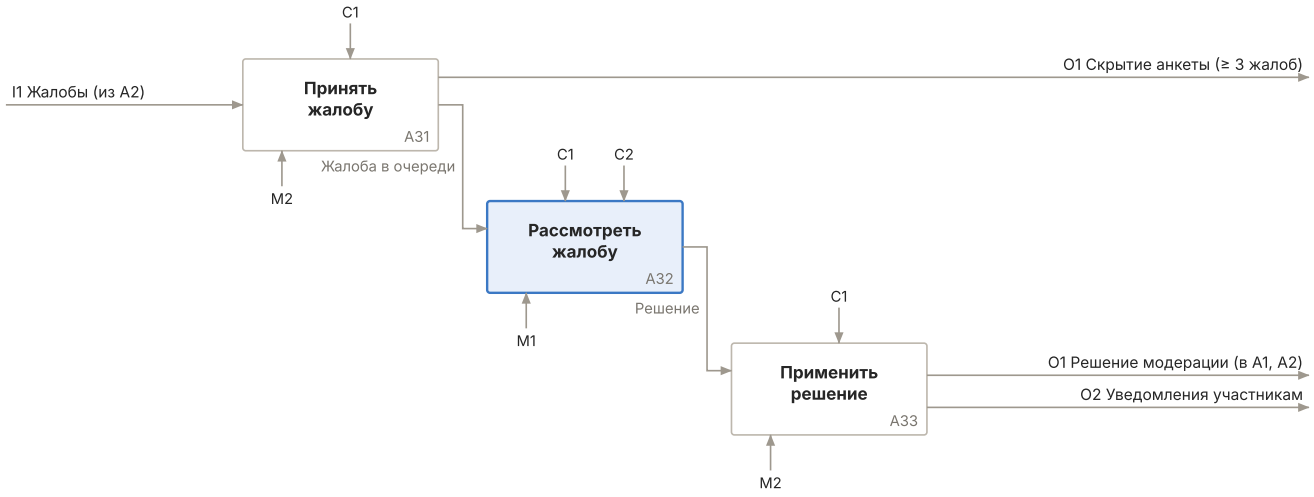
Логическое описание. Блок покрывает основной сценарий C2. A21 строит ленту из анкет (I1): в неё попадают только подтверждённые анкеты, без скрытых модератором (C2) и без студентов своего института, если они это запретили (C1). A22 обрабатывает лайк студента: проверяет лимит (C1), который снимает Premium (C3), и при взаимном лайке выдаёт совпадение. Оценённые анкеты возвращаются в A21 как управление — это обратная связь, чтобы одна анкета не показывалась дважды. Совпадение открывает чат в A23; из чата может уйти жалоба (O2). Переписка переходит в A24, где студенты согласуют встречу; итог — совпадения, переписка и встречи (O1).

Блок	Вход	Управление	Выход	Механизм
A21 Сформировать ленту	I1 Анкеты	C1, C2, оценённые анкеты (из A22)	Лента анкет	M1
A22 Обработать лайки	Лента, I2 лайк	C1 (30 в сутки), C3 Premium	Совпадение, оценённые анкеты	M1
A23 Вести переписку	Совпадение, I2 сообщение	C1	Переписка, O2 Жалобы	M1
A24 Согласовать встречу	Переписка, I2 предложение	C1 (публичное место)	O1 Встречи	M1

6.3. Декомпозиция A3 «Обеспечивать безопасность»

Решение по жалобе принимает модератор, система только скрывает анкету до проверки

IDEF0 · декомпозиция A3 «Обеспечивать безопасность»



IDEF0 A3 · декомпозиция «Обеспечивать безопасность»

Узел A3 · I1 — жалобы (из A2) · C1 — правила модерации · C2 — 152-ФЗ · O1 — решение модерации (в A1, A2) · O2 — уведомления · M1 — модератор · M2 — сервер приложения

Логическое описание. Блок покрывает сценарий C3. A31 принимает жалобу и ставит её в очередь; если на анкету пожаловались три разных пользователя (C1), анкета сразу скрывается из ленты (O1) — до решения человека. A32 — единственная работа модератора (M1): он видит жалобу и только ту переписку, на которую пожаловались (C2 ограничивает доступ к персональным данным), и принимает решение. A33 применяет решение: меняет статус пользователя (O1, уходит в A1 и A2) и уведомляет автора жалобы и нарушителя (O2).

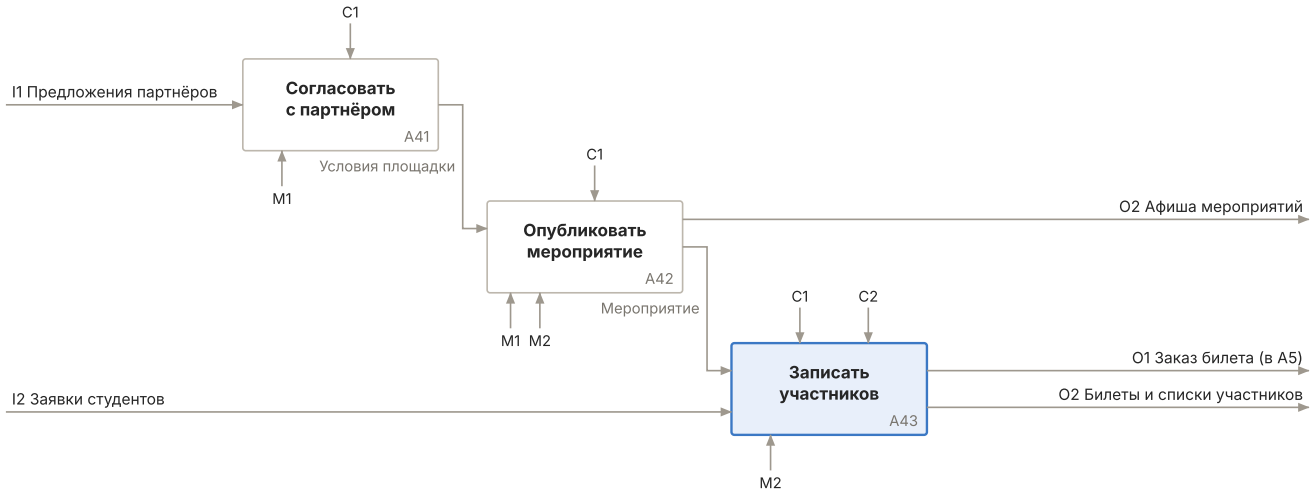
Блок	Вход	Управление	Выход	Механизм
A31 Принять жалобу	I1 Жалоба	C1 (3 жалобы → скрыть)	Жалоба в очереди, O1 скрытие	M2
A32 Рассмотреть жалобу	Жалоба в очереди	C1, C2 (доступ только к переписке по жалобе)	Решение	M1

Блок	Вход	Управление	Выход	Механизм
A33 Применить решение	Решение	C1	O1 Решение модерации, O2 Уведомления	M2

6.4. Декомпозиция A4 «Проводить мероприятия»

Место в списке участников закрепляется только после подтверждения оплаты

IDEF0 · декомпозиция A4 «Проводить мероприятия»



IDEF0 A4 · декомпозиция «Проводить мероприятия»

Узел A4 · I1 — предложения партнёров · I2 — заявки студентов · C1 — правила (18+, вместимость) · C2 — статус оплаты (из A5) · O1 — заказ билета (в A5) · O2 — афиша, билеты и списки участников · M1 — организатор · M2 — сервер приложения

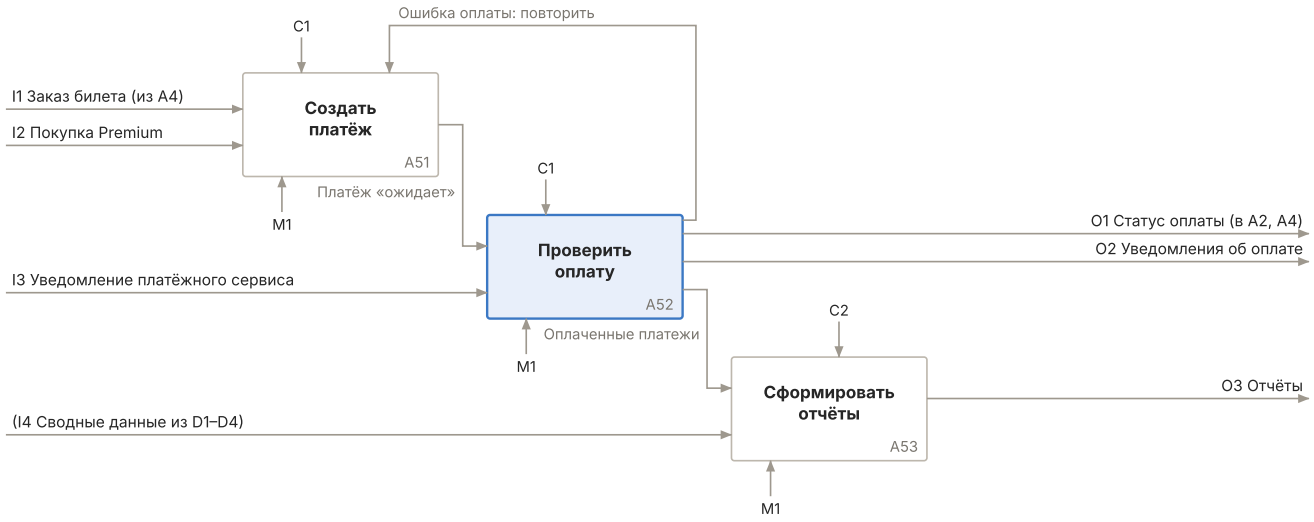
Логическое описание. Блок покрывает сценарий C4 и первую половину C5. Организатор (M1) согласует с партнёром дату, вместимость и цену (A41) и публикует мероприятие (A42), которое попадает в афишу (O2). A43 принимает заявку студента и проверяет вместимость (C1): если места есть, создаётся бронь и заказ билета уходит на оплату (O1). Бронь становится билетом только по статусу оплаты из A5 (C2) — так информация возвращается со второго уровня ветви 5 обратно в ветвь 4.

Блок	Вход	Управление	Выход	Механизм
A41 Согласовать с партнёром	I1 Предложения партнёров	C1	Условия площадки	M1
A42 Опубликовать мероприятие	Условия площадки	C1	Мероприятие, O2 Афиша	M1, M2
A43 Записать участников	Мероприятие, I2 Заявки	C1 (вместимость, 18+), C2 Статус оплаты	O1 Заказ билета, O2 Билеты и списки	M2

6.5. Декомпозиция A5 «Принимать и проверять оплату»

Только подтверждённая платёжным сервисом оплата меняет статус билета и Premium

IDEF0 · декомпозиция A5 «Принимать и проверять оплату»



IDEF0 A5 · декомпозиция «Принимать и проверять оплату»

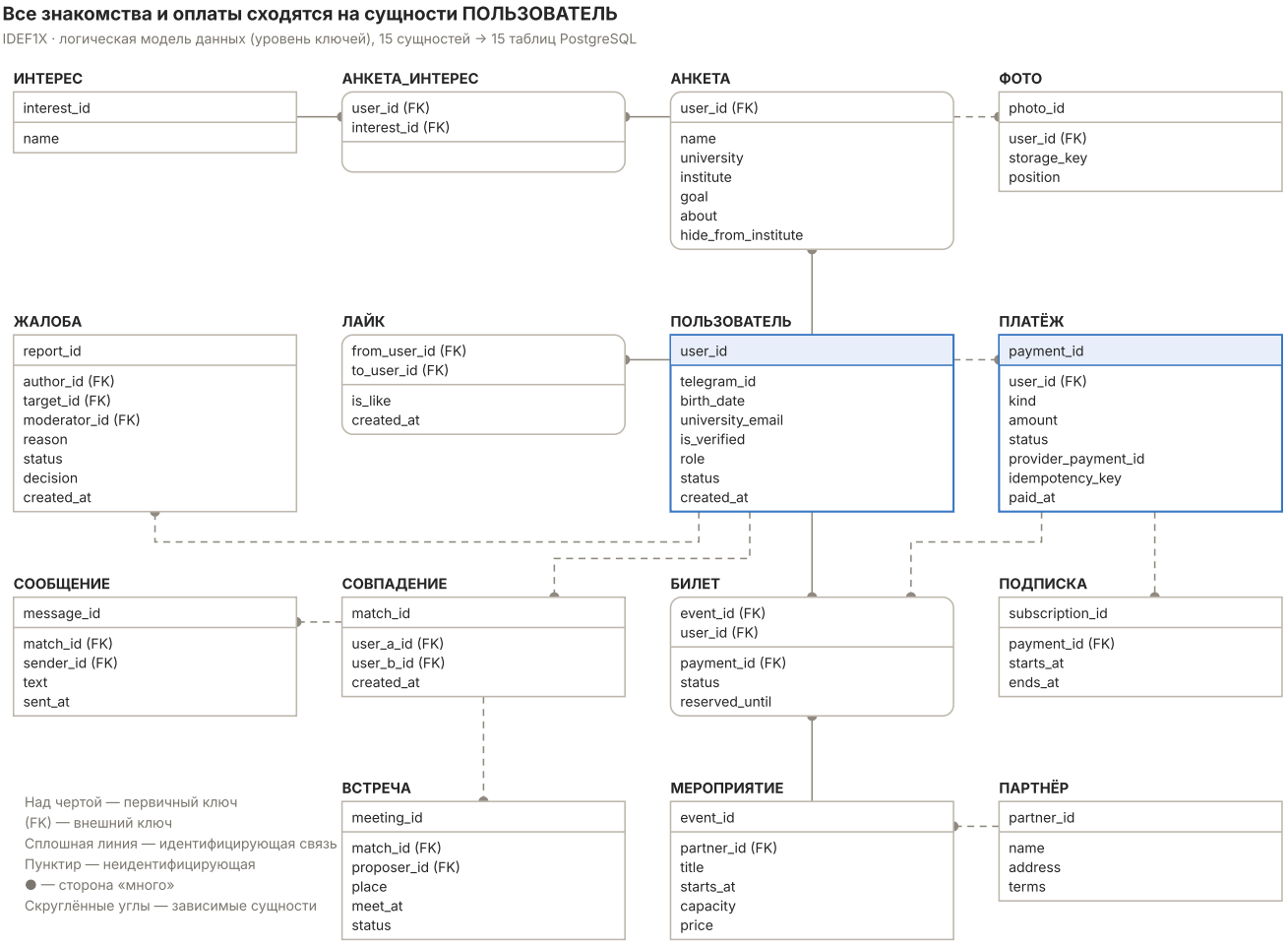
Узел A5 · I1 — заказ билета (из A4) · I2 — покупка Premium · I3 — уведомление платёжного сервиса · (I4) — сводные данные о пользователях, знакомствах и мероприятиях из D1–D4 (туннельная стрелка) · C1 — тарифы (349 ₽, бронь 30 минут) · C2 — 152-ФЗ · O1 — статус оплаты (в A2, A4) · O2 — уведомления об оплате · O3 — отчёты · M1 — сервер приложения

Логическое описание. Блок покрывает сценарии C5–C7 и выполняет последнюю задачу деятельности — проверку оплаты. A51 по тарифу (C1) создаёт платёж со статусом «ожидает» и получает ссылку на оплату. A52 сравнивает платёж с уведомлением платёжного сервиса (I3): при успехе выдаёт статус оплаты (O1), который уходит в A4 и A2, и уведомление студенту (O2). При ошибке платёж возвращается в A51 на повторную оплату — обратная связь по управлению. Проверка оплаты завершает обработку платежа: A43 получает результат и подтверждает билет. A53 стоит после A52 потому, что считает выручку только по уже проверенным платежам, но в цепочку обработки платежа не входит: он отдельно берёт сводные данные за период из D1–D4 (I4) — регистрации, совпадения, удержание D7, мероприятия — и выпускает отчёты (O3) только в обезличенном виде (C2).

Блок	Вход	Управление	Выход	Механизм
A51 Создать платёж	I1 Заказ билета, I2 Покупка Premium	C1 Тарифы, ошибка оплаты (из A52)	Платёж «ожидает»	M1
A52 Проверить оплату	Платёж, I3 Уведомление	C1 (бронь 30 мин)	O1 Статус оплаты, O2 Уведомления, оплаченные платежи	M1
A53 Сформировать отчёты	Оплаченные платежи, (I4) сводные данные D1–D4	C2 (обезличенные данные)	O3 Отчёты	M1

7. Модель данных IDEF1X

IDEF1X — расширение IDEF1 для реляционных баз данных: модель показывает сущности, их ключи и связи. Имена атрибутов записаны так, как они будут называться в таблицах PostgreSQL.



IDEF1X · модель данных

Логическое описание. В модели 15 сущностей: 11 независимых (прямые углы) и 4 зависимые (скруглённые углы) — АНКЕТА, АНКЕТА_ИНТЕРЕС, ЛАЙК и БИЛЕТ: их первичный ключ состоит из внешних ключей родителей, поэтому без родителей запись существовать не может.

- **ПОЛЬЗОВАТЕЛЬ → АНКЕТА** — связь «один к одному»: ключ анкеты совпадает с ключом пользователя. В пользователе хранятся статус (активен, скрыт, заблокирован — его меняет модерация), почта вуза и признак is_verified. Признак лежит именно в пользователе, а не в анкете: студент подтверждает почту (A12) до того, как заполнит анкету (A13), и лента (A21) допускает анкету, только если у её владельца is_verified = true.
- **АНКЕТА ↔ ИНТЕРЕС** — «многие ко многим», разрешена ассоциативной сущностью АНКЕТА_ИНТЕРЕС.
- **ЛАЙК** — зависимая сущность с составным ключом (from_user_id, to_user_id): повторный лайк той же анкеты невозможен на уровне базы. При встречном лайке создаётся СОВПАДЕНИЕ (user_a_id, user_b_id).

- **СОВПАДЕНИЕ → СООБЩЕНИЕ, ВСТРЕЧА** — переписка и встречи существуют только внутри совпадения; sender_id и proposer_id — роли пользователя, ссылающиеся на одного из участников пары.
- **ЖАЛОБА** ссылается на пользователя в трёх ролях: автор, нарушитель и модератор.
- **ПАРТНЁР → МЕРОПРИЯТИЕ → БИЛЕТ ← ПОЛЬЗОВАТЕЛЬ** — билет зависит от мероприятия и студента; ссылка на ПЛАТЁЖ появляется после оплаты.
- **ПЛАТЁЖ → ПОДПИСКА** — подписка Premium создаётся только из оплаченного платежа; ключ идемпотентности защищает от повторной обработки уведомления.

Хранилища DFD (раздел 8) соответствуют таблицам модели: так выполняется правило «БД = хранилища DFD».

Хранилище DFD	Таблицы (сущности IDEF1X)	Задачи дерева, которые пишут
D1.1 Пользователи	ПОЛЬЗОВАТЕЛЬ	11, 12, 33
D1.2 Анкеты	АНКЕТА, АНКЕТА_ИНТЕРЕС, ИНТЕРЕС	13
D1.3 Фото	ФОТО (+ файлы в объектном хранилище)	13
D2.1 Лайки и совпадения	ЛАЙК, СОВПАДЕНИЕ	22
D2.2 Сообщения	СООБЩЕНИЕ	23
D2.3 Встречи	ВСТРЕЧА	24
D3 Жалобы	ЖАЛОБА	31, 32
D4.1 Партнёры / D4.2 Мероприятия / D4.3 Билеты	ПАРТНЁР / МЕРОПРИЯТИЕ / БИЛЕТ	41 / 42 / 43
D5.1 Платежи / D5.2 Подписки	ПЛАТЁЖ / ПОДПИСКА	51, 52 / 52

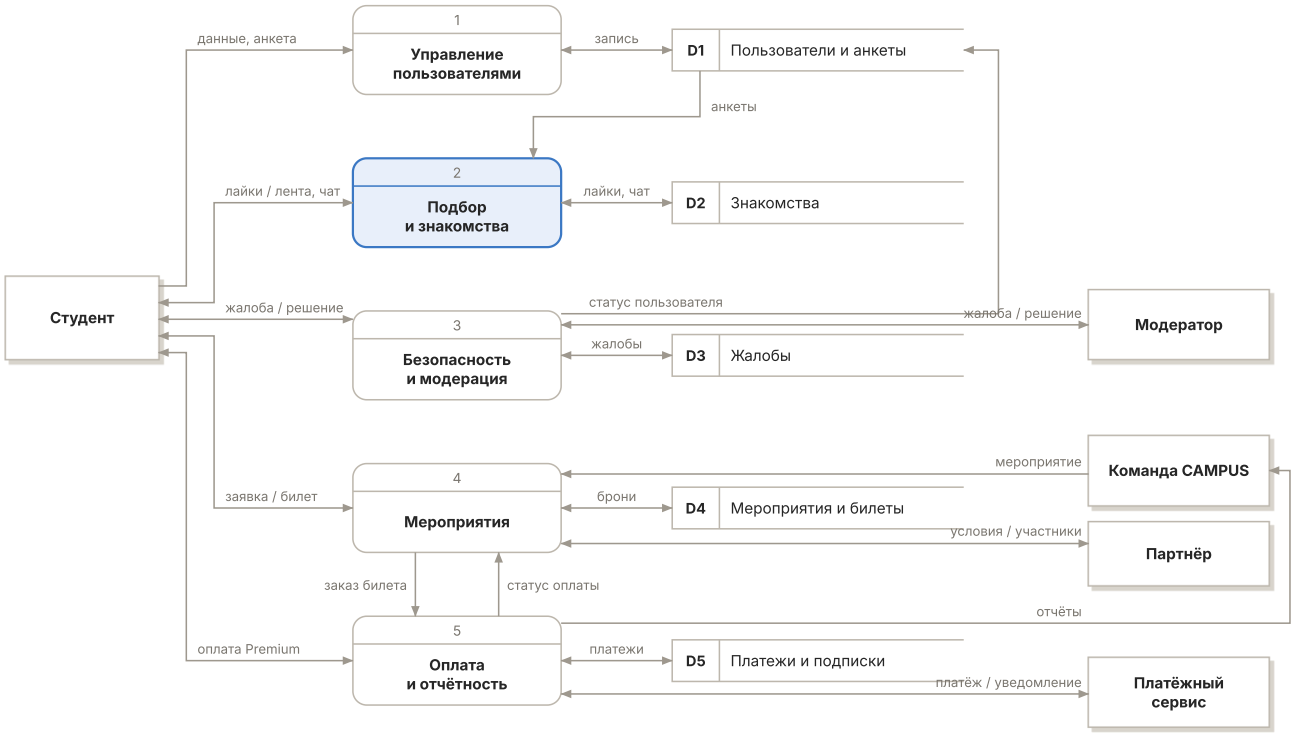
8. Диаграммы потоков данных (DFD)

DFD показывает, какие данные движутся между процессами, внешними сущностями и хранилищами. Процессы 1–5 соответствуют блокам A1–A5, процессы 11–53 — листьям дерева; хранилища — таблицам модели IDEF1X. На первом уровне хранилища показаны укрупнённо (D1–D5), на втором — разделены на таблицы (D1.1, D1.2 …).

8.1. DFD первого уровня

Студент работает со всеми процессами, а данные между ними передаются через хранилища

DFD · процессы первого уровня (нотация Гейна–Сарсона), хранилища показаны укрупнённо

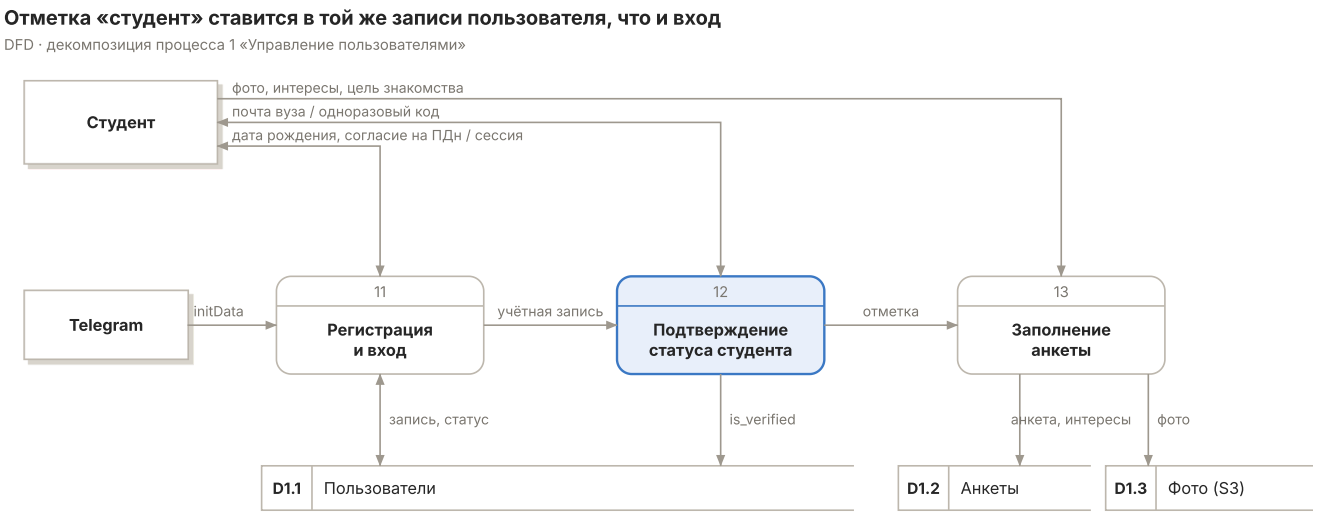


DFD · диаграмма потоков данных первого уровня · скруглённый блок — процесс, прямоугольник с тенью — внешняя сущность, открытый прямоугольник — хранилище

Логическое описание. Внешних сущностей пять: студент, модератор, команда CAMPUS, партнёр и платёжный сервис. Студент взаимодействует со всеми пятью процессами, поэтому стоит слева, а остальные сущности — справа, рядом со «своими» процессами. Процесс 1 хранит пользователей и анкеты в D1; процесс 2 читает анкеты из D1 и пишет лайки, совпадения и сообщения в D2. Процесс 3 пишет жалобы в D3 и возвращает в D1 статус пользователя — это единственная запись процесса в чужое хранилище, и именно она реализует петлю обратной связи A3 → A1. Процессы 4 и 5 обмениваются заказом билета и статусом оплаты; платёжный сервис присылает уведомления, а команда получает отчёты.

Из диаграммы видно, что самый нагруженный по данным процесс — «Подбор и знакомства»: он читает анкеты, пишет лайки и сообщения при каждом действии студента. Поэтому таблицы D2 получают индексы по пользователю, а лента отдаётся постранично.

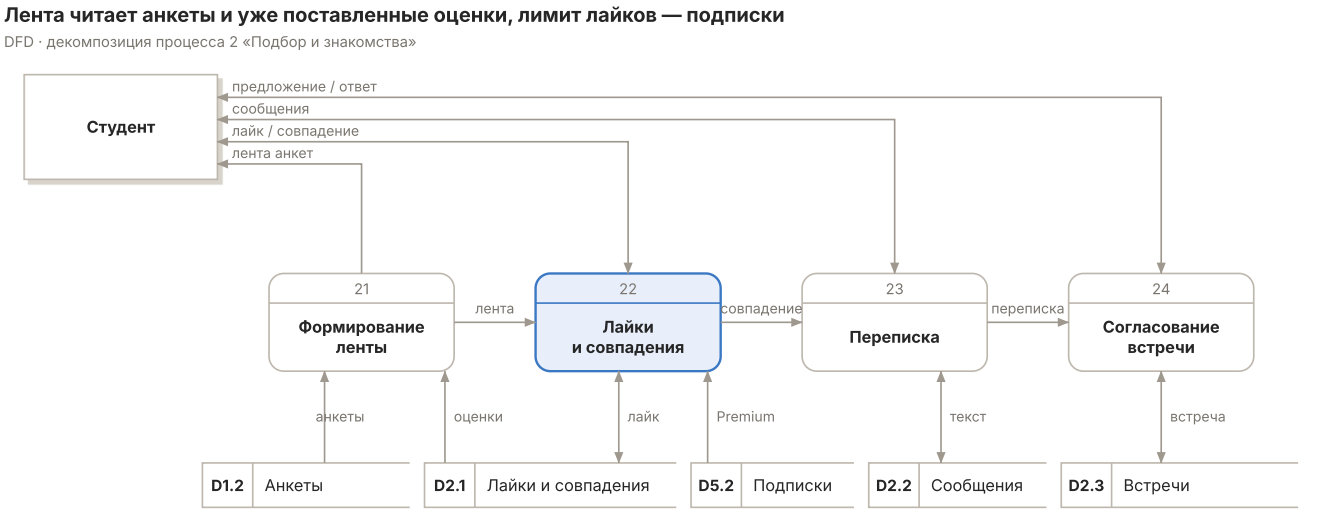
8.2. DFD процесса 1 «Управление пользователями»



DFD · декомпозиция процесса 1

Логическое описание. Telegram передаёт подписанные данные входа (initData), студент — дату рождения и согласие на обработку ПДн. Процесс 11 создаёт или находит запись в D1.1 и проверяет статус (заблокированный не войдёт). Процесс 12 отправляет одноразовый код на почту вуза и после проверки ставит в той же записи D1.1 отметку is_verified. Процесс 13 сохраняет анкету и интересы в D1.2, а файлы фото — в объектное хранилище D1.3.

8.3. DFD процесса 2 «Подбор и знакомства»



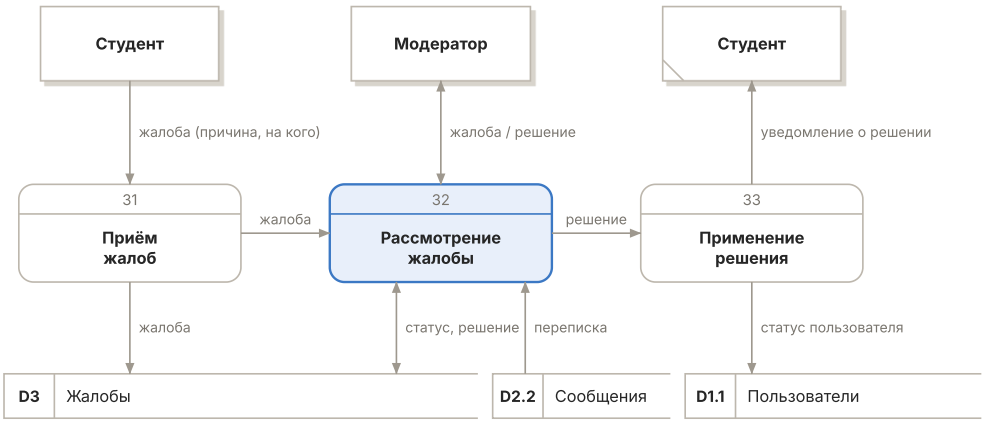
DFD · декомпозиция процесса 2

Логическое описание. Процесс 21 собирает ленту: читает анкеты из D1.2 и уже поставленные оценки из D2.1, чтобы не показывать одну анкету дважды; анкеты скрытых и заблокированных пользователей в ленту не попадают. Процесс 22 записывает лайк в D2.1 и читает D5.2, чтобы узнать, действует ли у студента Premium и есть ли лимит. При взаимном лайке создаётся совпадение. Процесс 23 хранит сообщения в D2.2, процесс 24 — предложения и ответы о встрече в D2.3.

8.4. DFD процесса 3 «Безопасность и модерация»

Модератор видит только переписку, на которую пожаловались

DFD · декомпозиция процесса 3 «Безопасность и модерация»



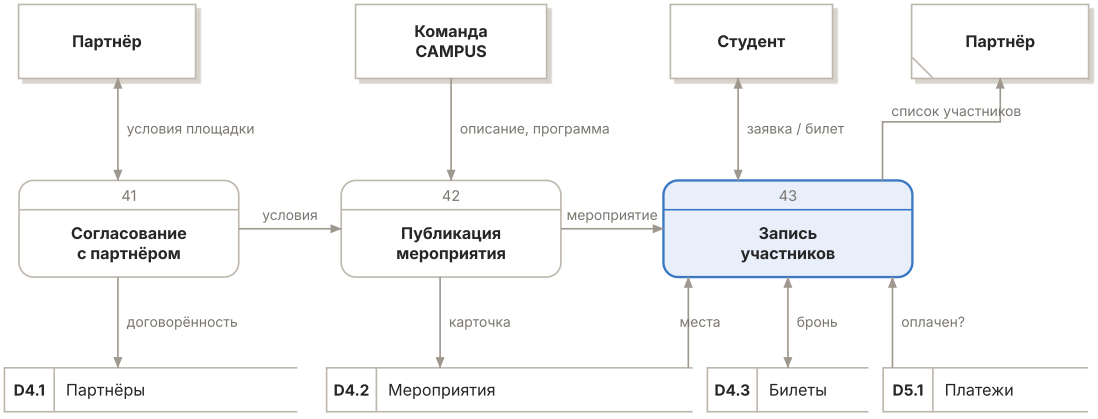
DFD · декомпозиция процесса 3 · срезанный угол — повторное изображение внешней сущности

Логическое описание. Студент отправляет жалобу в процесс 31, который записывает её в D3. Процесс 32 показывает жалобу модератору и читает из D2.2 только ту переписку, на которую пожаловались. Решение записывается в D3 и передаётся процессу 33, который меняет статус пользователя в D1.1 и отправляет уведомление (сущность «Студент» нарисована второй раз, чтобы не тянуть линию через всю диаграмму).

8.5. DFD процесса 4 «Мероприятия»

Билет подтверждается по статусу платежа из хранилища D5.1

DFD · декомпозиция процесса 4 «Мероприятия»



DFD · декомпозиция процесса 4

Логическое описание. Партнёр и организатор согласуют условия площадки (процесс 41, хранилище D4.1). Процесс 42 публикует карточку мероприятия в D4.2. Процесс 43 принимает заявку студента, проверяет число свободных мест в D4.2 и создаёт бронь в D4.3. Бронь становится билетом, когда в D5.1 появляется оплаченный платёж; после этого партнёр получает список участников.

8.6. DFD процесса 5 «Оплата и отчётность»

Платёж создаётся до оплаты, а подтверждается только уведомлением сервиса

DFD · декомпозиция процесса 5 «Оплата и отчётность»



DFD · декомпозиция процесса 5

Логическое описание. Процесс 51 читает бронь из D4.3 (или запрос Premium от студента), создаёт платёж «ожидает» в D5.1 и запрашивает у платёжного сервиса ссылку на оплату. Процесс 52 получает уведомление (webhook), находит платёж в D5.1 по его идентификатору и меняет статус; для Premium создаёт запись в D5.2. Процесс 53 читает платежи и агрегаты из D1–D4 и отдаёт команде отчёт за период.

9. Модели IDEF3

IDEF3 показывает порядок действий внутри сценария: сценарии C1, C2 и C5 из раздела 3 расписаны как декомпозиция блоков A1, A2, A4–A5 (расширенный уровень — не меньше двух диаграмм). Обозначения: прямоугольник — единица работы (UOW) с номером, J — перекрёсток (& — все ветви, X — ровно одна), ELAB — пояснение. Двойная черта слева — сходящийся перекрёсток, справа — расходящийся.

9.1. Сценарий C1 «Регистрация и подтверждение» (декомпозиция A1)



IDEF3 · регистрация и подтверждение статуса студента (A1)

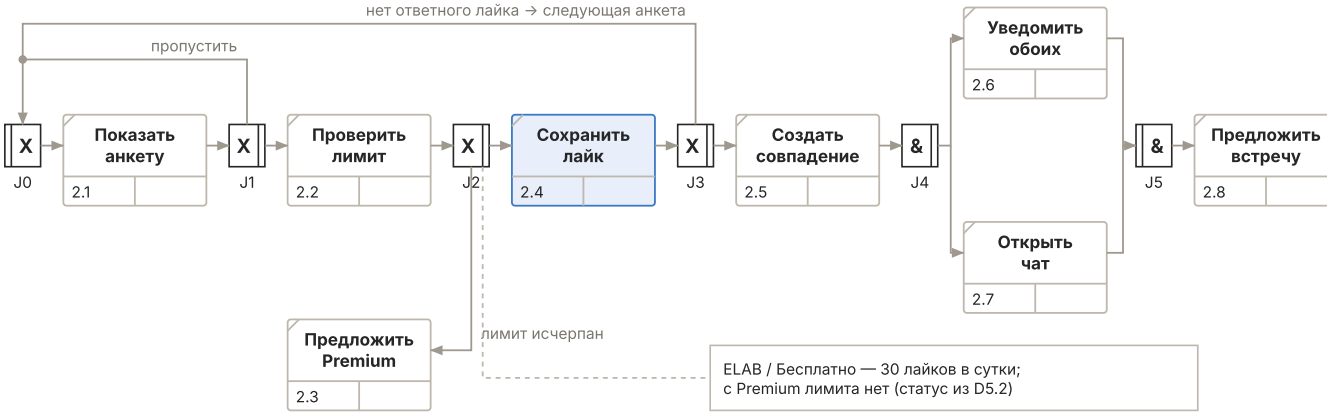
Логическое описание. После входа через Telegram перекрёсток J1 проверяет возраст: младше 18 — отказ (1.2). Студент вводит почту вуза и получает код (1.4). Если код неверный, J3 возвращает процесс через J2 к повторной отправке — не больше трёх раз. После верного кода фото и интересы заполняются параллельно (J4 и J5 — &), и анкета публикуется только когда готовы обе работы.

Перекрёсток	Тип	Смысл
J1	X, расходящийся	18 и старше → ввод почты; младше → отказ (1.2)
J2	X, сходящийся	Отправка кода: первая или повторная после ошибки
J3	X, расходящийся	Код верный → J4; неверный → возврат в J2
J4	&, расходящийся	Фото и интересы заполняются обе
J5	&, сходящийся	Публикация анкеты (1.8) — когда готовы обе работы

9.2. Сценарий C2 «Знакомство» (декомпозиция A2)

Совпадение возникает только при взаимном лайке, иначе студент возвращается к ленте

IDEF3 · сценарий C2 «Знакомство» · декомпозиция блока A2

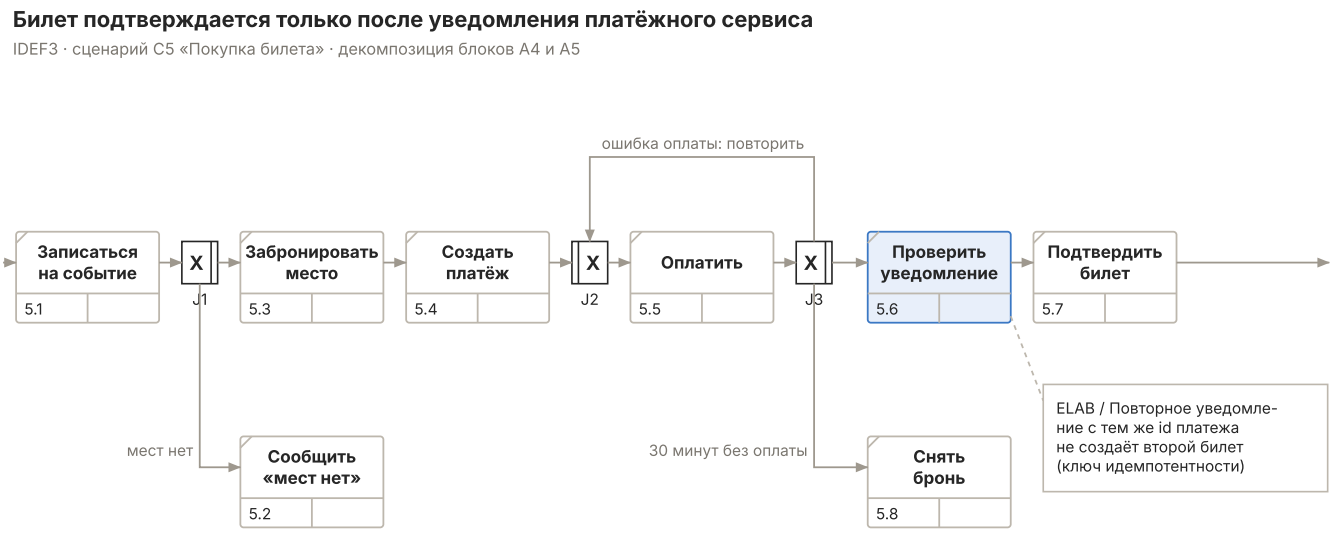


IDEF3 · от лайка до предложения встречи (A2)

Логическое описание. Студент смотрит анкету (2.1). Перекрёсток J1 разводит действия: «пропустить» возвращает к следующей анкете через J0, «лайк» ведёт к проверке лимита (2.2). Если лимит исчерпан, J2 предлагает Premium (2.3, дальше работает сценарий C6). Лайк сохраняется (2.4), и J3 проверяет встречный лайк: если его нет — следующая анкета, если есть — создаётся совпадение (2.5). Уведомление обоих и открытие чата идут параллельно, после чего можно предложить встречу (2.8).

Перекрёсток	Тип	Смысл
J0	X, сходящийся	Показ анкеты: первый или следующий после пропуска / невзаимного лайка
J1	X, расходящийся	Пропустить → J0; лайк → проверка лимита
J2	X, расходящийся	Лимит есть → сохранить лайк; исчерпан → предложить Premium
J3	X, расходящийся	Встречный лайк есть → совпадение; нет → J0
J4 / J5	&, расходящийся / сходящийся	Уведомление и открытие чата выполняются обе

9.3. Сценарий C5 «Покупка билета» (декомпозиция A4 и A5)



IDEF3 · покупка билета и проверка оплаты (A43, A51, A52)

Логическое описание. Студент записывается на мероприятие (5.1); если мест нет, J1 завершает сценарий сообщением (5.2). Иначе место бронируется на 30 минут (5.3) и создаётся платёж (5.4). После оплаты J3 разводит три исхода: успех ведёт к проверке уведомления (5.6) и подтверждению билета (5.7), ошибка возвращает к оплате через J2, а если 30 минут прошли без оплаты — бронь снимается (5.8).

Перекрёсток	Тип	Смысл
J1	X, расходящийся	Места есть → бронь; нет → сообщение «мест нет»
J2	X, сходящийся	Оплата: первая попытка или повтор после ошибки
J3	X, расходящийся	Успех → проверка; ошибка → J2; 30 минут без оплаты → снять бронь

10. Требования FURPS+ и сценарии качества

Требования к ИС сгруппированы по модели FURPS+: Functionality, Usability, Reliability, Performance, Supportability и «+» — ограничения. Каждое требование проверяемо и привязано к сценарию (раздел 3) и блоку модели; числа взяты из предположений о нагрузке.

10.1. Требования

ID	Категория	Требование	Сценарий	Блок
F1	Functionality	Вход через Telegram Mini App с проверкой подписи initData; отказ младше 18	C1	A11
F2	Functionality	Подтверждение статуса студента кодом на почту в домене вуза	C1	A12
F3	Functionality	Анкета: 1–6 фото, 3–10 интересов, цель знакомства, «скрыть от своего института»	C1	A13
F4	Functionality	Лента: только подтверждённые анкеты, без оценённых, скрытых и заблокированных	C2	A21
F5	Functionality	Лайк и пропуск; 30 лайков в сутки без Premium; совпадение при взаимном лайке	C2	A22
F6	Functionality	Чат только внутри совпадения; предложение встречи с местом и временем	C2	A23, A24
F7	Functionality	Жалоба из профиля и чата; скрытие при 3 жалобах; решение модератора	C3	A31–A33
F8	Functionality	Мероприятие с вместимостью и ценой; запись с бронью на 30 минут	C4, C5	A41–A43
F9	Functionality	Оплата билета и Premium в тестовом режиме, подтверждение только по уведомлению	C5, C6	A51, A52
F10	Functionality	Отчёт за период: регистрации, совпадения, удержание D7, оплаты	C7	A53
U1	Usability	От открытия Mini App до первой анкеты в ленте — не больше 5 шагов	C1	A1
U2	Usability	Интерфейс для телефона (от 360 px), светлая и тёмная тема Telegram	—	A1–A5
U3	Usability	Понятные ошибки: «неверный код, осталось 2 попытки», «лимит лайков исчерпан»	C1, C2	A12, A22
R1	Reliability	Повторное уведомление об оплате не создаёт второй платёж или билет (ключ идемпотентности)	C5	A52
R2	Reliability	Сообщение не теряется при обрыве связи: клиент повторяет отправку с тем же id	C2	A23
R3	Reliability	Резервная копия базы раз в сутки, хранение 7 дней	—	M3
P1	Performance	Ответ API ленты ≤ 300 мс для 95% запросов при 60 одновременных пользователях	C2	A21

ID	Категория	Требование	Сценарий	Блок
P2	Performance	Новое сообщение доходит до собеседника не дольше чем за 2 с	C2	A23
S1	Supportability	Запуск всего сервиса одной командой (Docker Compose)	—	M3
S2	Supportability	Изменения схемы базы только через миграции	—	M3
S3	Supportability	Логи запросов и ошибок; автотесты API для сценариев C1–C5	—	A1–A5

«+» — ограничения:

- **Реализация:** Telegram Mini App (TypeScript, React), backend NestJS (TypeScript), PostgreSQL, Redis для одноразовых кодов, S3-совместимое объектное хранилище в РФ.
- **Интерфейсы:** REST API в формате JSON; Telegram Bot API для уведомлений; API платёжного сервиса в тестовом режиме.
- **Физические:** один сервер 2 vCPU / 4 ГБ и хранилище фото от 5 ГБ — из расчёта нагрузки в разделе 3.2.
- **Юридические:** согласие на обработку персональных данных при регистрации (152-ФЗ); хранение данных граждан РФ на серверах в РФ; удаление аккаунта по запросу; только 18+; точная геолокация не собирается; в отчётах — обезличенные данные.

10.2. Сценарии атрибутов качества

Нефункциональные требования расписаны как сценарии из шести частей: источник → стимул → среда → артефакт → отклик → мера отклика.

Сценарий	Источник	Стимул	Среда	Артефакт	Отклик	Мера отклика	Треб.
КС1. Вечерний пик	Студенты	60 человек листают ленту	21:00–23:00	A21, API	Лента отдаётся страницами по 20 анкет по индексу	≤ 300 мс для 95%	P1
КС2. Повторный webhook	Платёжный сервис	То же уведомление пришло дважды	Обычный режим	A52	Второе уведомление распознаётся по id платежа и игнорируется	0 двойных билетов	R1
КС3. Подбор кода	Злоумышленник	10 попыток ввести код подряд	Обычный режим	A12	После 3 ошибок код аннулируется, пауза 15 мин	0 подобранных кодов	F2, U3
КС4. Массовые жалобы	Студенты	3 жалобы на одну анкету за час	Обычный режим	A31	Анкета скрыта из ленты до решения	скрытие ≤ 1 мин, решение ≤ 24 ч	F7
КС5. Новый вуз	Команда	Нужно подключить второй вуз	Разработка	A12, C1	Домен вуза добавляется в настройки, код не меняется	≤ 1 дня работы	S1, S2

11. Модели UML

Диаграммы построены по спецификации UML 2.5.1 консорциума OMG; международный стандарт ISO/IEC 19505 соответствует UML 2.4.1 и с ней совместим. UML дополняет IDEF: IDEF0 и IDEF3 описывают деятельность, а UML — саму ИС: кто ей пользуется, какие данные она хранит и как её части взаимодействуют.

Диаграмма UML	Вид	Что показывает	Связь с другими моделями
Вариантов использования	Поведенческая	Актёров и функции системы с их точки зрения	Актёры — столбцы матрицы ответственности; варианты — листья дерева
Классов	Структурная	Классы предметной области, атрибуты и связи	Совпадает с моделью IDEF1X
Последовательности (три диаграммы)	Поведенческая (взаимодействие)	Обмен сообщениями между частями системы во времени	Сценарии C1, C2, C5; детализация IDEF3

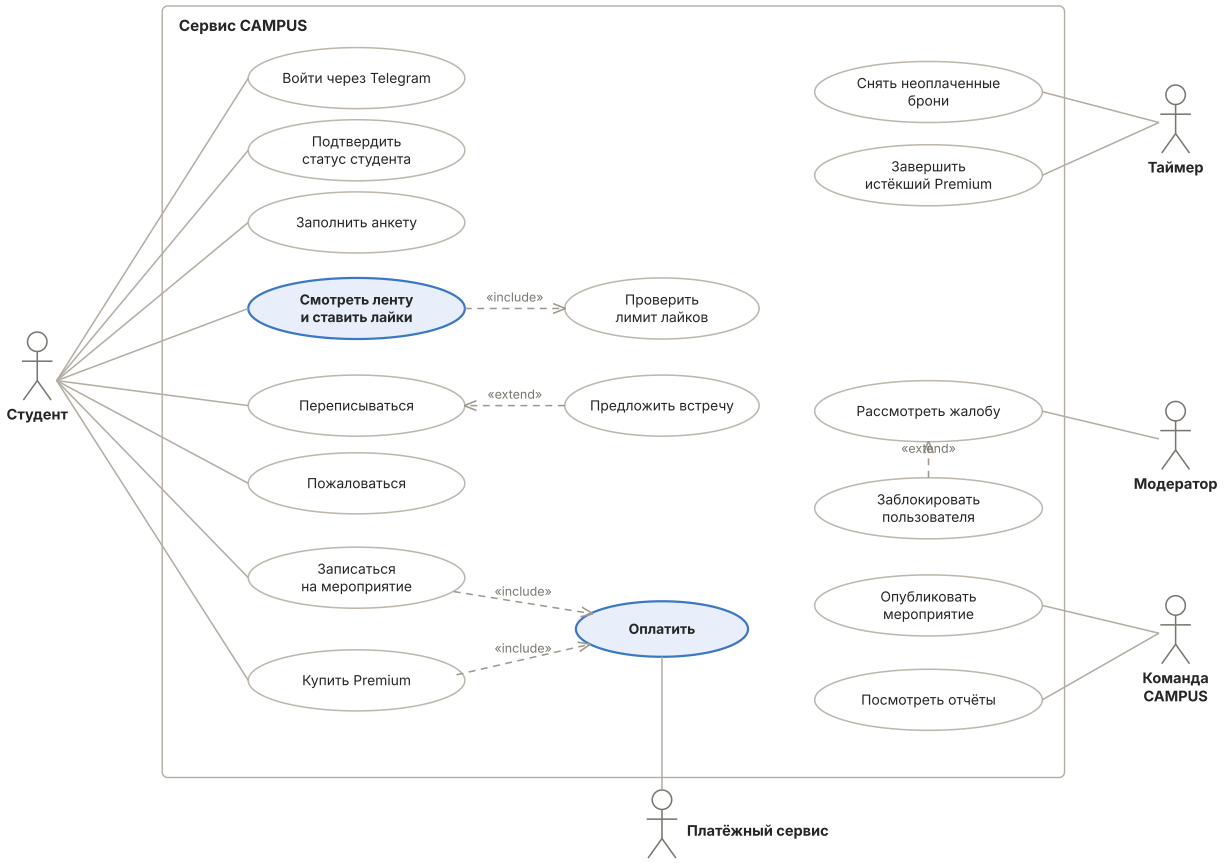
Правила нотации, которые соблюдены:

- Актёры стоят вне границы системы; варианты использования названы глаголом и дополнением.
- «include» — включаемое поведение выполняется всегда, стрелка идёт от базового варианта; «extend» — по условию, стрелка идёт к базовому варианту.
- В классах указаны видимость (+), имя и тип атрибута; у каждой ассоциации — кратность на обоих концах; закрашенный ромб — композиция.
- В диаграммах последовательности синхронный вызов — сплошная линия с закрашенной стрелкой, ответ — пунктир; условия — комбинированный фрагмент alt со сторожевыми условиями в квадратных скобках.

11.1. Диаграмма вариантов использования

Студент — главный актёр; оплата всегда включает проверку платёжным сервисом

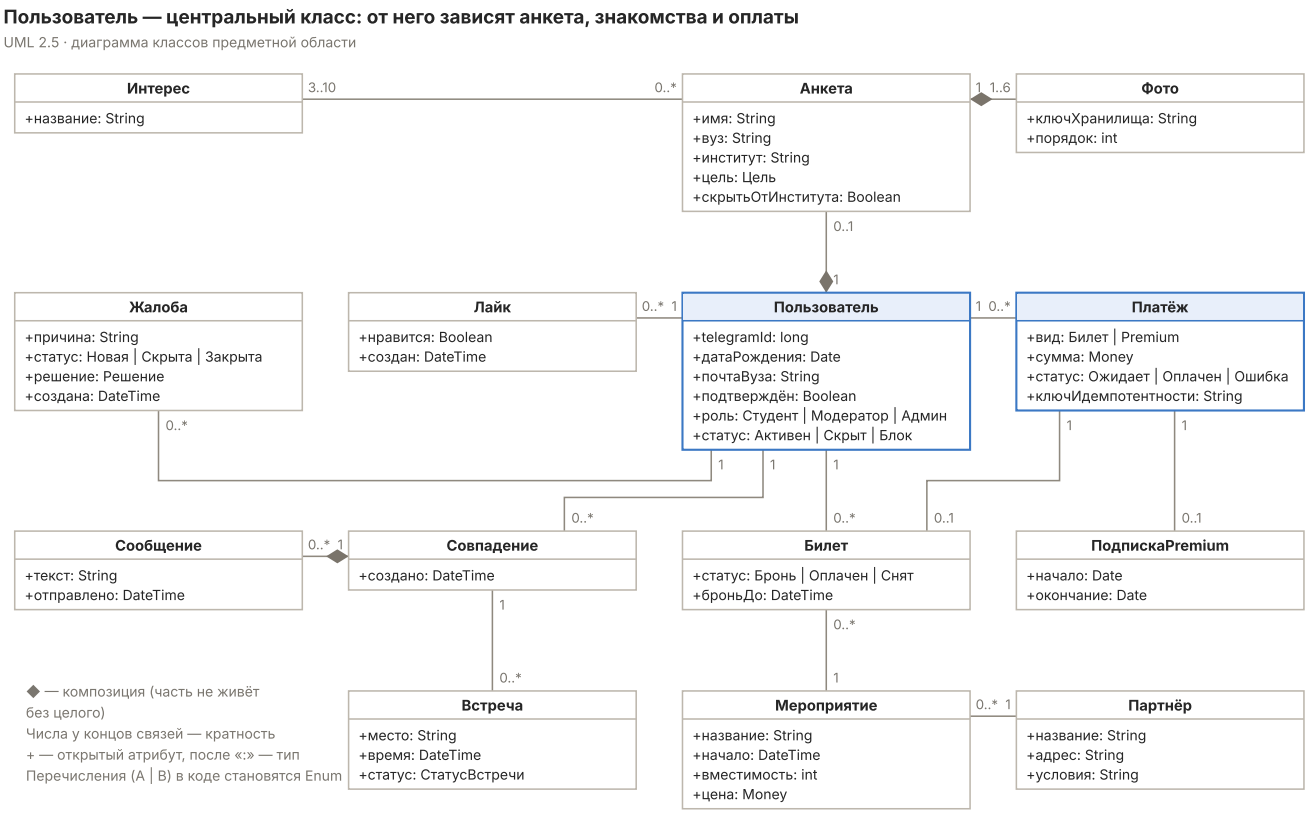
UML 2.5 · диаграмма вариантов использования



UML · диаграмма вариантов использования

Логическое описание. Актёров пять: студент, модератор, команда CAMPUS, платёжный сервис и таймер — системный актёр, который по расписанию снимает неоплаченные брони и завершает истёкший Premium. Они взяты из столбцов матрицы ответственности, где стоит буква «И». «Смотреть ленту и ставить лайки» всегда включает проверку лимита, а запись на мероприятие и покупка Premium всегда включают оплату («include»). «Предложить встречу» расширяет переписку, а «Заблокировать пользователя» — рассмотрение жалобы: они выполняются только при условии («extend»). Каждый вариант использования — будущий экран или эндпоинт программы.

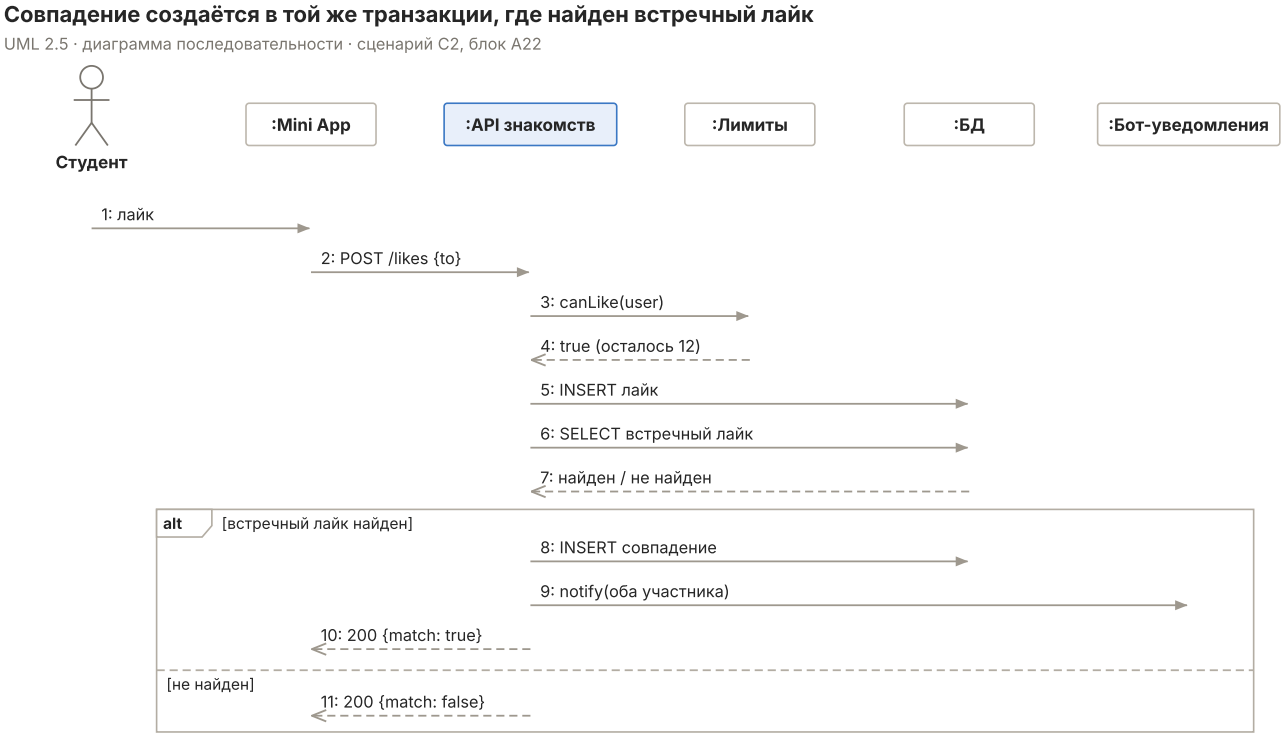
11.2. Диаграмма классов



UML · диаграмма классов

Логическое описание. Классы соответствуют сущностям модели IDEF1X; ассоциативная сущность АНКЕТА_ИНТЕРЕС на диаграмме классов стала связью «многие ко многим» между Анкетой и Интересом (3..10 интересов у анкеты). Центральный класс — Пользователь: в нём хранятся почта вуза и признак «подтверждён» (как в IDEF1X), у него одна Анкета (композиция — анкета удаляется вместе с пользователем), лайки, совпадения, жалобы, билеты и платежи. Фото входят в Анкету композицией (1..6), Сообщения — в Совпадение. Перечисления (статусы, роль, вид платежа) задают допустимые значения прямо в модели, а в коде становятся Enum.

11.3. Диаграмма последовательности: лайк и совпадение (C2)



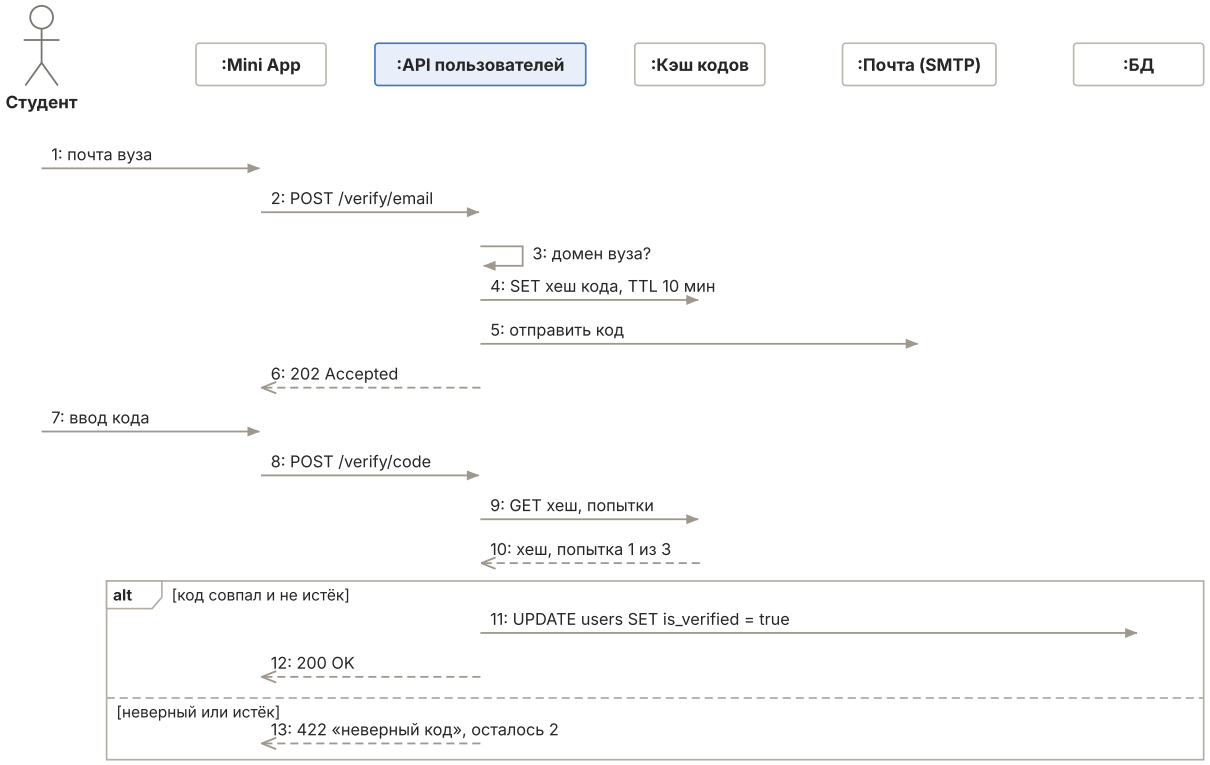
UML · диаграмма последовательности, сценарий C2

Логическое описание. Студент ставит лайк, Mini App вызывает API знакомств (1–2). API спрашивает у сервиса лимитов, можно ли ставить лайк (3–4), записывает лайк и ищет встречный (5–7). Фрагмент alt даёт две ветви: если встречный лайк найден, создаётся совпадение, бот уведомляет обоих и клиент получает match: true (8–10); если нет — match: false (11). Это тот же сценарий, что IDEF3 в разделе 9.2, но на уровне запросов между частями системы.

11.4. Диаграмма последовательности: подтверждение статуса студента (C1)

Отметка «студент» появляется только после проверки одноразового кода

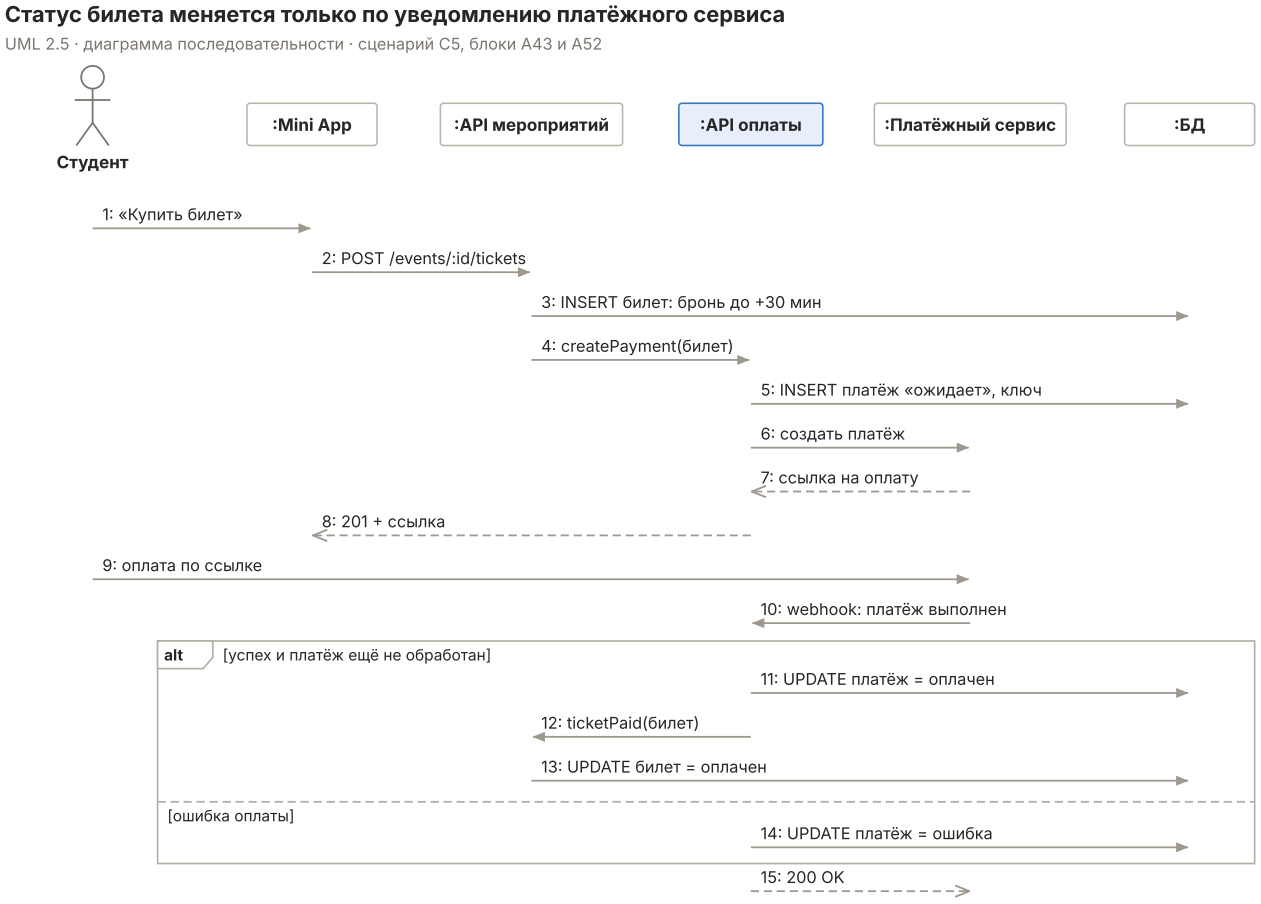
UML 2.5 · диаграмма последовательности · сценарий C1, блок A12



UML · диаграмма последовательности, сценарий C1

Логическое описание. API проверяет, что домен почты принадлежит вузу (3), сохраняет в кэше только хеш кода со сроком 10 минут (4) и отправляет код письмом (5). При вводе кода API сравнивает хеши и считает попытки (8–10). Если код совпал, в записи студента в таблице users (хранилище D1.1) ставится is_verified = true (11–12) — анкеты ещё может не быть, её студент заполнит следующим шагом (A13); иначе клиент получает ответ 422 с числом оставшихся попыток (13). Сам код нигде не хранится в открытом виде.

11.5. Диаграмма последовательности: покупка билета (C5)



UML · диаграмма последовательности, сценарий C5

Логическое описание. Студент нажимает «Купить билет»: API мероприятий создаёт бронь на 30 минут (1–3) и просит API оплаты создать платёж (4). Платёж записывается со статусом «ожидает» и ключом идемпотентности, платёжный сервис возвращает ссылку (5–8). Студент платит (9), сервис присылает webhook (10). Фрагмент alt: если оплата успешна и платёж ещё не обработан, статус платежа и билета меняется на «оплачен» (11–13); при ошибке платёж помечается ошибкой (14). В любом случае сервис получает 200 OK (15), чтобы не присылать уведомление снова.

12. Словарь стрелок и выводы

12.1. Словарь стрелок (диаграмма A0)

Стрелка	Тип	Откуда → куда	Содержание
I1 Данные студента	I	граница → A1	Telegram-аккаунт, дата рождения, почта вуза, фото, интересы
I2 Действия студентов	I	граница → A2, A4, A5	Лайки, сообщения, жалобы, заявки, покупка Premium
I3 Предложения партнёров	I	граница → A4	Площадка, даты, вместимость, условия
I4 Уведомления платёжного сервиса	I	граница → A5	Результат оплаты (webhook)
C1 Правила сервиса и модерации	C	граница → A1–A4	18+, лимит лайков, правила общения и мероприятий
C2 152-ФЗ «О персональных данных»	C	граница → A1, A3, A5	Согласие, хранение в РФ, обезличенные отчёты
C3 Тарифы и условия оплаты	C	граница → A5	Premium 349 ₽ на 30 дней, бронь 30 минут
Анкета, статус	O→I	A1 → A2	Подтверждённая анкета и статус пользователя
Жалобы	O→I	A2 → A3	Жалоба: автор, нарушитель, причина
Решение модерации	O→C (обратная связь)	A3 → A1, A2	Статус: активен, скрыт, заблокирован
Заказ билета	O→I	A4 → A5	Бронь: мероприятие, студент, сумма
Статус оплаты	O→C (обратная связь)	A5 → A2, A4	Оплачен / ошибка / бронь снята; Premium до даты
O1 Совпадения, переписка, встречи	O	A2 → граница	Результат знакомства для студента
O2 Мероприятия и билеты	O	A4 → граница	Афиша, билеты, списки участников
O3 Уведомления	O	A3, A5 → граница	О решении модератора и об оплате
(Сводные данные D1–D4)	I, туннельная	хранилища D1–D4 → A5 (A53)	Регистрации, совпадения, встречи, мероприятия — для отчёта
O4 Отчёты	O	A5 → граница	Регистрации, совпадения, D7, оплаты за период
M1 Модератор	M	граница → A3	Рассмотрение жалоб
M2 Организатор мероприятий	M	граница → A4	Согласование и публикация мероприятий

Стрелка	Тип	Откуда → куда	Содержание
M3 Сервер приложения	M	граница → A1–A5	Mini App, NestJS, PostgreSQL, объектное хранилище

12.2. Выводы

1. Деятельность сервиса распадается на пять ветвей (A1–A5) — это пять модулей будущей программы: пользователи, знакомства, модерация, мероприятия и оплата; 16 листьев второго уровня дают список эндпоинтов.
2. Две петли обратной связи — решение модерации (A3 → A1, A2) и статус оплаты (A5 → A2, A4) — повторяются в IDEF0, DFD, IDEF3 и диаграммах последовательности; последний блок деятельности — проверка оплаты.
3. Каждый лист дерева пишет в своё хранилище; хранилища DFD совпадают с таблицами IDEF1X и классами UML (15 таблиц, из них 4 зависимые).
4. Матрица ответственности показала, что студент исполняет 9 задач из 16, а задачи 21, 52 и 53 выполняются автоматически, — отсюда актёры диаграммы вариантов использования.
5. Нагрузка пилота небольшая (≈ 60 пользователей онлайн в пик), всё помещается на один сервер; главный риск — не производительность, а плотность аудитории и безопасность пользователей.

Приложение А. Техническое задание для разработчиков

ТЗ сформировано по модели: модули — блоки А1–А5, эндпоинты — листья дерева, таблицы — сущности IDEF1X, требования — FURPS+. Документ предназначен для двух разработчиков и рассчитан на MVP за 4 недели.

А.1. Назначение и пользователи

Telegram Mini App для знакомств студентов одного вуза (пилот — УрФУ) с чатом, предложениями встреч, мероприятиями и тестовой оплатой. Роли и права — по матрице ответственности (раздел 1.3): студент, модератор, администратор (он же организатор).

А.2. Модули, эндпоинты и таблицы

Модуль (блок)	Эндпоинты REST API	Таблицы	Задачи
Пользователи (А1)	POST /auth/telegram · POST /verify/email · POST /verify/code · GET, PUT /profile · POST /profile/photos	users, profiles, photos, interests, profile_interests	11–13
Знакомства (А2)	GET /feed · POST /likes · GET /matches · GET, POST /matches/:id/messages · POST /matches/:id/meetings · PATCH /meetings/:id	likes, matches, messages, meetings	21–24
Модерация (А3)	POST /reports · GET /admin/reports · POST /admin/reports/:id/decision	reports, users.status	31–33
Мероприятия (А4)	GET /events · POST /admin/partners · POST /admin/events · POST /events/:id/tickets	partners, events, tickets	41–43
Оплата и отчёты (А5)	POST /payments · POST /payments/webhook · GET /admin/stats?from&to	payments, subscriptions	51–53

А.3. Требования и ограничения

- Функциональные, качественные и юридические требования — таблица FURPS+ (раздел 10); каждое требование закрывается автотестом или ручной проверкой по сценарию.
- Стек: React + TypeScript (Telegram Mini App), NestJS, PostgreSQL, Redis, S3-совместимое хранилище в РФ, Docker Compose.
- Безопасность: проверка подписи initData на сервере; секреты — только в переменных окружения. Доступ проверяется на сервере, а не скрытием в интерфейсе, по двум отдельным правилам: (1) фото и анкету видит тот, кому эта анкета разрешена в ленте: владелец подтверждён, не скрыт и не заблокирован, а зритель не из его института, если включено «скрыть от своего института»; (2) переписку и предложения встреч видят только два участника конкретного совпадения (и модератор — только переписку, на которую пожаловались).
- Фоновые задачи (таймер): каждые 5 минут снимать брони старше 30 минут; раз в сутки завершать истёкший Premium.

А.4. Этапы работ

Неделя	Результат	Задачи дерева
1	Каркас проекта, схема БД и миграции, вход через Telegram, деплой на сервер	11
2	Подтверждение почты, анкета и фото, лента и лайки	12, 13, 21, 22
3	Совпадения, чат, встречи, жалобы и решение модератора	23, 24, 31–33
4	Мероприятия, тестовая оплата, отчёт, исправления и демонстрация	41–43, 51–53

А.5. Критерии приёмки

- Два тестовых студента проходят сценарии C1 и C2 до предложения встречи без вмешательства разработчиков.
- Жалоба от трёх пользователей скрывает анкету; решение модератора меняет статус пользователя (C3).
- Повторное уведомление об оплате не создаёт второй билет; неоплаченная бронь снимается через 30 минут (C5).
- Выполнены требования P1 и R1–R3; сервис поднимается одной командой.